sfizz for MPC OS

User manual · version 1.0.3



The sfizz 1.2.3 SFZ sampler as a native instrument plugin for Gen1 MPC OS hardware, on a one-page screen modelled on sfizz's own editor. Play SFZ libraries straight from SD cards and USB drives, streamed from disk, with key switches, the instrument's own controls on knobs, Scala tunings and high-quality interpolation.

Contents

1. Requirements
2. Installing and uninstalling
3. Quick start
4. The screen
5. Loading instruments
6. Controls and MIDI
7. Settings reference
8. Tuning
9. Projects
10. Performance and sound quality
11. Troubleshooting
12. Credits and licences

sfizz is by the sfztools contributors; this MPC port is a community project, not an official sfztools release, and is not affiliated with inMusic or Akai. MPC and Force are trademarks of their owners.

1. Requirements

- **A Gen1 MPC OS device** (32-bit ARM): Force, MPC Live, Live II, One, X or Key 61. Gen2 devices (MPC Live III, 64-bit) are not supported by this release; the installer refuses them.
- **MPC OS 3.x.** The screen uses the 3.x skin format.
- **Root shell access (SSH)** to the device. Stock MPC OS doesn't offer this: the device must be modded. The installer edits MPC's settings file to register the plugin.
- **SFZ instruments** on an SD card, a USB drive or in the plugin's own library folder. sfizz reads WAV, FLAC, Ogg Vorbis, AIFF and WavPack samples.

> Installing plugins this way is unofficial. Back up your projects and settings first, and use it at your own risk.

2. Installing and uninstalling

Scripted install (recommended)

1. Download `sfizz-1.0.3-mpc-armv7.zip` from the project's GitHub Releases page and unzip it on your computer.
2. Copy the whole folder to the device:

   ```
   scp -r sfizz-1.0.3 root@<device-ip>:/tmp/
   ```

3. Run the installer:

   ```
   ssh root@<device-ip> sh /tmp/sfizz-1.0.3/install.sh
   ```

The installer checks the device, asks for confirmation, **stops MPC** (save your project first), copies the plugin folder to `/sdcard/Synths/sfztools - VST - sfizz/`, backs up `MPC.settings`, adds sfizz to MPC's plugin list and starts MPC again. `-y` skips the question; `-t <folder>` installs into another Synths folder (for example on a card). Running it again upgrades in place.

Manual install

The zip's `INSTALL.md` lists the manual steps: copy the plugin folder, stop MPC, back up and edit `MPC.settings`, start MPC.

Uninstall

```
ssh root@<device-ip> sh /tmp/sfizz-1.0.3/uninstall.sh
```

This removes the plugin folder and its plugin-list entry, and restarts MPC. SFZ files you put in the plugin's `sfizz/Instruments` folder are kept. Projects that used sfizz load without it.

3. Quick start

1. Copy an SFZ instrument folder to an SD card or USB drive (keep the `.sfz` file next to its samples folder) and insert it. A good free one: *Salamander Grand Piano V3* (github.com/sfzinstruments/SalamanderGrandPiano).

2. On a track, add **sfizz** from the plugin browser (Instrument plugins). A new sfizz plays a simple sine.
3. Tap the ⌄ button at the top right of the file box. The browser opens.
4. Tap your card, then folders, then the `.sfz` file. The name, counts and memory update when it has loaded.
5. Play. Turn the Q-Links: they control Volume, Pan and the instrument's own controls.
6. Tap ⌄ again to close the browser.

4. The screen

sfizz's desktop editor has four tabs. On the MPC they are side by side on one page: the header, then **Information**, **Controls** and **Settings**, then a keyboard you can play.



The screen with Salamander Grand Piano V3 loaded.

1. **File name** of the loaded instrument (without `.sfz`). Shows *Loading…* while loading.
2. ‹› load the previous / next SFZ file in the same folder.
3. ⌄ opens and closes the SFZ browser.
4. **Key switch**: the current key switch's note (badge) and name. Instruments without key switches show *No key switch*.
5. **Voices** sounding now · **Max** voices (tap to choose) · **Memory** used by sample data in RAM.
6. **Volume** and **Pan**: MIDI CC 7 and CC 10 for the instrument (0–127; pan 64 is centre).
7. **Information**: the instrument's curves, masters, groups, regions and samples.
8. **Controls**: up to eight of the instrument's own controllers, named as the instrument names them, with their CC number.
9. **Settings**: engine options, rendering quality and tuning (section 7).
10. **Keyboard**: two octaves and a C. Keys light while they sound, from MIDI (pads, a controller, the sequencer) or from a tap. Tap a key to play it (velocity 100, for 0.6 s; the screen reports a touch, not a finger lifting, so a tap can't be held). Tap a white key below the black keys. ‹› shift it by an octave; the C keys show their octave (middle C is C4, as sfizz numbers them). Taps play sfizz directly: they don't reach MPC's sequencer, so they aren't recorded. The octave is saved with the project.

> Not on the MPC screen, compared with the desktop editor: the output meters, the Theme and Zoom options, the "when freewheeling" quality settings (MPC never freewheels) and the file menu's Edit, Create and Open folder entries.

5. Loading instruments



The browser covers the Information and Controls panels while it is open.

The browser

- ⌂ (home) lists the places: **Plugin Library** (the plugin's own `sfizz/Instruments` folder), every SD card and USB drive, and `sdcard` (the internal storage).
- Tap a folder (shown with a trailing `/`) to open it, and the **folder-up** button to go back. A tap acts once: a second touch within a third of a second of a folder opening is ignored, so one tap never opens two things.
- Tap an SFZ file to load it. The loaded file is lit green. The browser stays open so you can try others.
- ∧ ∨ page through long folders, eight rows at a time; the counter shows where you are.
- The bottom line reports the result: the number of regions and samples, or why something didn't load.

Where to put instruments

- **On a card or drive**: any folder. Keep each instrument as its author ships it: the `.sfz` file finds its samples by their path relative to it.
- **In the plugin library**: `/sdcard/Synths/sfztools - VST - sfizz/sfizz/Instruments/`. The installer keeps this folder when you upgrade or uninstall.

Loading takes a moment

sfizz reads the start of every sample into memory (the *preload*) and streams the rest from disk while you play. A large library can take several seconds to load from a card. The plugin is silent while it loads; notes played during a long load are skipped.

Instruments that use the extended SFZ format

sfizz supports SFZ v1 and v2 and many ARIA extensions (key switches, CC curves, release triggers, crossfades, effects). Opcodes it doesn't support are ignored, as on the desktop.

6. Controls and MIDI

Q-Links

Bank	Q-Links
Bank 1	Volume, Pan, Controls 1–6
Bank 2	Controls 7–8, Frequency, Stretch, Sample quality, Oscillator quality

A Q-Link on an empty Controls slot does nothing. Settings that reload samples (Max voices, Oversampling, Preload size) are not on Q-Links, so a turn can't interrupt playing.

The Controls panel

sfizz lists the MIDI controllers the instrument uses. The panel shows up to eight of them: first those the instrument names (its `label_cc` lines), then the others by number. CC 7 and CC 10 are left out because they are the Volume and Pan knobs. A controller that doesn't fit still works from MIDI.

MIDI

- **Every controller reaches the instrument**, including CC 20–35 (which some MPC plugins keep for Q-Links). Knobs follow the controllers they show.
- Pitch bend, channel and polyphonic aftertouch, and the sustain pedal (CC 64) go to sfizz.
- All Sound Off (CC 120) and All Notes Off (CC 123) silence the instrument.
- The plugin plays on any MIDI channel the track sends.

7. Settings reference

Setting	Values (default in bold)	What it does
Max (voices)	1, 2, 4, 8, 16, 32, **64**, 96, 128, 160, 192, 224, 256	How many voices can sound at once. Lower it if the device runs out of CPU with dense playing.
Volume	0–127 (**the instrument's default**, often 100)	MIDI CC 7 to the instrument.
Pan	0–127 (**64**, centre)	MIDI CC 10 to the instrument.
Preload size	4, 8, 16, **32**, 64, 128, 256 kB	How much of each sample sfizz keeps in memory; the rest streams from disk. Raise it if a slow card makes note starts drop out; lower it to save memory. Changing it reloads the samples.
Oversampling	**1x**, 2x, 4x, 8x	Upsamples the samples when they load. It can clean up heavily pitched sounds, but multiplies memory and load time. Changing it reloads the samples.
Sustain cancels release	**Off**, On	sfizz's option of the same name: decides whether holding the sustain pedal cancels release samples, as some other SFZ players do.
Sample (quality)	Nearest, Linear, Polynomial, **Sinc 8**, Sinc 12, 16, 24, 36, 48, 60, 72	How samples are resampled when played at another pitch or sample rate. Higher is cleaner and costs more CPU (section 10).
Oscillator (quality)	Nearest, **Linear**, High, Dual-High	The same for sfizz's built-in waveforms (`*sine` , `*saw` and wavetables).
Scala file	**DefaultScale** (equal temperament), files in `sfizz/Scales`	Tap to step to the next scale; × goes back to equal temperament.
Root key	note **C** · octave **4**	The key the scale starts on (C4 = MIDI note 60).
Frequency	300–500 Hz (**440.0**)	Tuning of A4. Drag the value or turn its Q-Link; the ⌄ next to it picks a historical pitch (Baroque 415, French Law 435, European 442 …).
Stretch	0–100% (**0**)	Piano-style stretch tuning: octaves widen towards the ends of the keyboard.

8. Tuning

sfizz tunes with Scala (`.scl`) files, a common format for microtonal and historical scales; thousands are freely available online.

1. Copy `.scl` files to `/sdcard/Synths/sfztools - VST - sfizz/sfizz/Scales/`.
2. Tap the **Scala file** button to step through them (equal temperament first). Tap **×** to reset.
3. Set the **Root key** (note and octave) the scale is built on.
4. Set the reference pitch with **Frequency**, or pick a historical one from its ⌄ list.

The new scale list is read when you step, so files copied while MPC runs show up without a restart.

9. Projects

A saved project keeps, for each sfizz instance: the instrument file, the Scala file, every setting, and the positions of the Volume, Pan and Controls knobs. When the project loads, the instrument loads again in the background.

- Instruments are saved by their full path. If the file has moved (another card, a renamed drive), sfizz looks for a file of the same name in the plugin library, on every card and drive, and in `/sdcard`, up to four folders deep, and loads the first it finds.
- If it isn't found, the file name shows *(missing)* and the browser's status line says so. Load it again from the browser and save the project.

10. Performance and sound quality

The Gen1 MPC's processor is shared by every track and plugin. What costs CPU in sfizz, from most to least:

- **Voices sounding**: sustained piano chords with the pedal down can use dozens. Lower **Max** if needed.
- **Sample quality**: per voice, Sinc 8 costs about 10% more than Polynomial, Sinc 24 nearly 5× Polynomial, and the longer filters more still.
- **Oversampling**: more data per voice.

Loading samples while you play. Desktop sfizz reads the whole rest of a sample file the moment a key is hit, on a high-priority thread, and keeps it in memory with no limit; on the MPC that briefly maxed out the processor on every note with large libraries, and long sessions of chords could run out of memory. This port reads files gradually instead, a little ahead of what is playing, at normal priority, within a memory budget (a third of the memory free when the plugin starts, between 48 and 384 MB). Files no note uses are released after a few seconds. If the budget runs out (very many long notes at once), the notes that have played longest fade out quickly, as when sfizz runs out of voices, so new notes always sound. The sound is otherwise exactly the same.

Why Sinc 8 is the default: in offline measurements it removed about 24 dB of interpolation distortion compared with sfizz's desktop default (Polynomial) on bright samples pitched down, for a small extra cost. sfizz keeps samples in 32-bit float, so 24-bit libraries keep their full depth.

> If you hear crackles: lower Sample quality to Polynomial, lower Max voices, or check that other plugins on the project aren't already using most of the CPU. If note starts drop out on a slow card, raise Preload size.

11. Troubleshooting

Problem	What to do
sfizz isn't in the plugin browser	Run `install.sh` again and let MPC restart. Check the device is Gen1 and on MPC OS 3.x.
The screen is empty or wrong	MPC OS 2.x can't read this skin: update to 3.x.
An instrument loads but some notes are silent	Samples are missing: copy the whole instrument folder, keeping its layout. Check that sample file names match the `.sfz` exactly.
"Could not load …"	The `.sfz` couldn't be read or parsed. Try it in sfizz on a computer.
"No SFZ files in …" after ‹ ›	The folder has no `.sfz` files at its top level; open a subfolder with the browser.
A card doesn't appear in the browser	Insert it before opening the browser, then tap ⌂ to refresh the list.
A Controls knob is missing	The panel shows eight controllers; the others still work from MIDI.
Silence for a moment after changing Max, Oversampling or Preload size	Expected: these reload the engine.
Long held notes end early while playing many keys with a big library	The sample memory budget is full (it is set from the memory free when the plugin starts): the oldest notes give way to new ones. Close other memory-hungry plugins or unload unused project samples, then reload the project; or lower Max voices.

12. Credits and licences

- **sfizz** by the sfztools contributors, BSD-2-Clause (github.com/sfztools/sfizz). This port uses the 1.2.3 engine with one change, to how it reads sample files (the sound is unchanged); its libraries keep their own licences.
- **VST2 wrapper, skin tools and installer**: mpc-vst-plugins by sd88me and contributors, MIT.
- **Artwork**: drawn after sfizz's editor; the sfizz logo and emblem come from sfizz-ui (BSD-2-Clause). Roboto, Apache-2.0.
- **This port**: BSD-2-Clause. Source: github.com/gmorb/mpc-vst-sfizz.